

LD2-A Sparse HDR and Logical-Node Cloud Rendering Specification

Backend-neutral HDR details, deterministic node-cloud preparation, provenance, bounds, and render accounting

mdstats development specification

2026-07-20

Contents

1	Status and authority	1
2	Objective	2
3	Non-objectives	2
4	Scientific invariants	2
4.1	Logical-node coordinates	2
4.2	Highest-density region	2
4.3	Deterministic display thinning	2
4.4	Intensity normalization	3
5	Data model	3
6	Backend-neutral HDR API	4
7	Two-pass cloud preparation	4
7.1	Pass 1: exact counting	4
7.2	Pass 2: selected-node realization	4
8	Expanded-display replication	4
9	Renderer integration	4
10	Resource policy	5
11	Acceptance criteria	5
11.1	Scientific and selection equivalence	5
11.2	Geometry	5
11.3	Determinism and resources	5
11.4	Regression	5
12	Documentation and citations	6

1 Status and authority

This specification defines architecture gate **LD2-A** under the normative `mdstats` Dynamical Framework and Density Plotting Architecture Standard. It is implemented in `mdstats` 0.19.47a0 from the `mdstats` 0.19.46a0 LD1-B baseline, where atomic `local_sparse` fields already preserve the LD1-A scientific estimator exactly.

LD2-A authorizes logical-node cloud rendering for both dense and block-sparse fields. Sparse triangular isosurfaces remain deferred to LD2-B.

2 Objective

Deliver a backend-neutral preparation layer

ScalarField3D + PeriodicNodeFieldAccess $\rightarrow$ HDR details $\rightarrow$ deterministic logical-node selection $\rightarrow$ Cartesian node cloud $\rightarrow$

The sparse path must never materialize a full dense logical array.

3 Non-objectives

LD2-A does not implement:

1. sparse marching cubes or sparse triangular meshes;
2. framework-vertex or framework-edge sparse preparation;
3. automatic backend selection;
4. multilevel refinement;
5. statistical resampling or stochastic point thinning;
6. recomputation of density for expanded display images.

4 Scientific invariants

4.1 Logical-node coordinates

A selected node $\mathbf{n} = (i, j, k)$ is located at

$$\mathbf{f}_{ijk} = \left(\frac{i}{N_1}, \frac{j}{N_2}, \frac{k}{N_3} \right), \quad \mathbf{x}_{ijk} = \mathbf{f}_{ijk} H_d.$$

No half-grid displacement is permitted.

4.2 Highest-density region

For requested fraction q , all positive nodes satisfying

$$\rho_g \geq c_q$$

belong to the eligible cloud, where c_q is the field's scientific HDR threshold. Threshold ties are retained before any display-point limit is applied. The scientific HDR result remains independent of rendering limits.

4.3 Deterministic display thinning

Let M be the number of eligible nodes in global C-order logical-index order, and let $K = \min(M, K_{\max})$. If $K = M$, all nodes are retained. If $K < M$, the selected eligible ranks are exactly those produced by

```
np.linspace(0, M - 1, K, dtype=np.int64)
```

and the output remains in increasing logical-flat-index order. This preserves the historical dense cloud policy and makes dense and sparse selection byte-identical when their fields agree.

4.4 Intensity normalization

Returned marker intensities are

$$I_g = \frac{\rho_g}{\max_{h \in S} \rho_h},$$

where S is the final selected set. Thus $0 < I_g \leq 1$ and the historical dense rendering semantics are preserved.

5 Data model

```
@dataclass(frozen=True, slots=True)
class DensityCartesianBounds:
    minimum: NDArray[np.float64] # (3,)
    maximum: NDArray[np.float64] # (3,)

@dataclass(frozen=True, slots=True)
class DensityNodeCloudResources:
    scanned_stored_node_count: int
    eligible_node_count: int
    selected_point_count: int
    truncated: bool
    index_bytes: int
    value_bytes: int
    cartesian_bytes: int
    intensity_bytes: int
    estimated_peak_bytes: int
    trace_count: int

@dataclass(frozen=True, slots=True)
class DensityTraceProvenance:
    field_key: str
    label: str
    storage_backend: str
    source_provenance: DensitySourceProvenance
    requested_mass_fraction: float
    scientific_hdr_threshold: float
    achieved_mass_fraction: float
    eligible_node_count: int
    selected_point_count: int
    selection_policy: str
    display_replication: str
    image_shift: tuple[int, int, int]

@dataclass(frozen=True, slots=True)
class DensityNodeCloud3D:
    logical_indices: NDArray[np.int64] # (n, 3)
    cartesian_positions: NDArray[np.float64] # (n, 3)
    relative_intensities: NDArray[np.float64] # (n,)
    hdr_details: SparseHDRDetails
    bounds: DensityCartesianBounds
    resources: DensityNodeCloudResources
    provenance: DensityTraceProvenance
```

All arrays are defensive, C-contiguous, and read-only. All records are schema-versioned and canonically serializable.

6 Backend-neutral HDR API

ScalarField3D gains

```
def hdr_details(self, q: float) -> SparseHDRDetails: ...
```

The dense implementation uses the same descending-value and tie-inclusive rule as the sparse reference and block fields. `threshold_for_mass_fraction(q)` remains the compatibility projection of `hdr_details(q).threshold`.

7 Two-pass cloud preparation

`prepare_density_node_cloud` uses public node access only.

7.1 Pass 1: exact counting

Iterate stored nodes in global lexicographic order and compute:

- scanned stored-node count;
- eligible node count at the scientific threshold;
- exact selected-point count after the cloud limit;
- exact output-array byte counts.

No dense logical array is allocated.

7.2 Pass 2: selected-node realization

Iterate the field again and retain only the deterministic eligible ranks. Construct logical indices, values, Cartesian coordinates, normalized intensities, bounds, provenance, and exact resource accounting.

The implementation validates monotonic logical-node order and rejects duplicate or unsorted public iteration.

8 Expanded-display replication

A canonical cloud is prepared once. With `display_replication="match_graph"`, already prepared points are translated by

$$\mathbf{x}' = \mathbf{x} + \mathbf{m}H_d$$

for each deterministic primary-cell image shift $\mathbf{m}$ from an expanded periodic graph view. Density is not recomputed. Each image is one Plotly trace in the same legend group and retains the same scientific field identity.

`match_graph` is rejected for non-expanded graph views and for mesh rendering until LD2-B. Replicated point and trace counts are checked before trace creation.

9 Renderer integration

The framework-dynamics renderer:

1. permits `local_sparse` only when `render_mode="voxel_cloud"`;
2. prepares backend-neutral clouds before Plotly trace allocation;
3. enforces scene-wide point and trace budgets using exact counts;
4. creates one trace per requested display image;
5. records trace-indexed `DensityTraceProvenance`;
6. exposes cloud bounds and resource summaries in render metadata;
7. continues to reject sparse meshes with an LD2-B diagnostic.

The compatibility function `density_voxel_cloud_arrays` delegates to the new preparation layer and returns the same three arrays/threshold tuple as before.

10 Resource policy

The preparation API enforces:

```
max_points
max_workspace_bytes
```

The renderer additionally enforces:

```
max_density_render_points
max_density_traces
max_plotly_traces
```

Resource failures occur before Plotly trace creation. No scientific threshold, grid, bandwidth, or density values are changed to satisfy rendering limits.

11 Acceptance criteria

11.1 Scientific and selection equivalence

For dense and block-sparse fields representing the same values:

```
HDR threshold absolute difference <= 5e-12 * max(1, reference maximum)
achieved HDR mass-fraction difference <= 5e-13
logical selected-index arrays are byte-identical
relative intensity arrays are byte-identical
```

Required fixtures include orthogonal and LTA-primitive cells, face/edge/corner crossings, overlapping clouds, tied thresholds, truncation, and $\sigma = 0$.

11.2 Geometry

For every returned point,

$$\left\| \mathbf{x}_g - \left(\frac{i}{N_1}, \frac{j}{N_2}, \frac{k}{N_3} \right) H_d \right\|_2 \leq 10^{-12} L_{\text{ref}}.$$

Bounds equal the componentwise extrema of the returned Cartesian points. Expanded replicas differ from canonical positions by exactly $\mathbf{m}H_d$ within the same tolerance.

11.3 Determinism and resources

1. repeated preparations produce byte-identical indices and positions;
2. block shape does not change selected scientific nodes;
3. sparse preparation never calls dense conversion;
4. exact resource counts equal realized array sizes;
5. every point, workspace, and trace limit has a focused failure test;
6. serialization round trips preserve all scientific records exactly.

11.4 Regression

1. default dense scientific fields and meshes remain compatible with `mdstats 0.19.46a0`;
2. the historical dense cloud output remains byte-identical for canonical rendering;
3. framework sparse channels remain rejected until LD3;
4. sparse mesh rendering remains rejected until LD2-B.

12 Documentation and citations

No new external numerical algorithm is introduced. Highest-density-region thresholds continue to follow Hyndman (1996); CIC continues to follow Hockney and Eastwood (1988). Deterministic cloud thinning, provenance, bounds, replication, and resource accounting are project-specific mdstats policies.